KUBERNETES

Kubernetes is an open-source container orchestration technology for automating deployment, scaling, and managing containerized applications. Kubernetes was originally developed by Google and is now maintained by the **Cloud Native Computing Foundation (CNCF)**.

Kubernetes Architecture

Kubernetes Architecture

---

1. Master (Control Plane)

Kubernetes master mainly runs with four components:

---

1. API Server

- It is the main management point of the entire cluster.
- When we interact with the Kubernetes cluster using `kubectl` commands, we are actually communicating with the API server.
- The API server is the only Kubernetes component that connects to all other components.
- It is responsible for:
  - Authentication
  - Authorization
- It also implements a **watch mechanism** to detect changes.
- This allows components such as:
  - Scheduler
  - Controller Manager

    to interact with it and match the **desired state** with the **actual state**.

---

2. etcd

- etcd is a distributed, consistent key-value store.

Om-Tech

- It is used to store configuration data of the entire Kubernetes cluster.
- It represents the state of the cluster at any given time:
    - What nodes exist
    - What pods should be running
    - Which nodes they are running on

Command (backup etcd):

etcdctl snapshot save <backup-file>

---

3. Scheduler

- Watches for unscheduled pods.
- Assigns pods to nodes.
- Decides placement based on:
    - Resource availability
    - Affinity / anti-affinity rules
    - Other constraints

---

4. Controller Manager

- A daemon that runs continuous control loops (controllers).
- Watches cluster state via API server.
- Ensures:
    - Desired state = Actual state

Examples of controllers:

- Deployment
- Replication / ReplicaSet
- StatefulSet
- DaemonSet

---

2. Major Components of Worker Nodes

Node (Worker)

A node is a worker machine where containers inside pods are launched by Kubernetes.

Om-Tech

1. Container Runtime

- The underlying software used to run containers.
- Common runtimes:
  - Docker
  - CRI-O
  - rktlet

---

2. Kubelet

- Agent running on each node.
- Ensures containers are running as expected.
- Communicates with the API server.

---

3. Kube-Proxy

- Network proxy running on each node.
- Responsible for:
  - Exposing services to external hosts
  - Forwarding requests to correct containers
  - Basic load balancing

---

4. Pod

- The smallest deployable unit in Kubernetes.
- A pod can have:
  - One or more containers

Om-Tech

Manifest File (Fields of Manifest File)

A manifest file in **Kubernetes** defines Kubernetes resources using YAML.

---

1. apiVersion

- `apiVersion` allows us to define which version of the API a given resource will use.

Examples:

- `v1`
- `apps/v1`
- `autoscaling/v1`

Commands:

kubectl api-versions # To list API versions in Kubernetes
kubectl api-resources -o wide # To get detailed list of API resources
kubectl api-resources --api-group <api-group-name> # To check objects under a specific API group

---

2. kind

- Represents the type of Kubernetes object defined in the manifest.
- Each resource type belongs to a specific API version.
- It is important that `kind` and `apiVersion` match correctly.

Examples:

- Pod
- Deployment

---

3. metadata

- Contains identifying information about the object:
  - `name`
  - `namespace`
  - `labels`

---

4. spec

- Defines the desired state of the object.
- Includes properties such as:
  - Container image
  - Number of replicas

Om-Tech

---

5. Labels and Annotations

Labels

- Used to identify and group objects.
- Example:

```
labels:
 app: demo-app
```

Annotations

- Used to store additional metadata.
- Example:

```
annotations:
 created-by: jenkins-ci
```

---

6. Selectors

- Used to identify a set of objects using matching labels.
- Label selectors are the core grouping mechanism in Kubernetes.

---

a) Equality-based Selectors

- Filter based on key-value pairs.
- Operators:
  - `=`
  - `==`
  - `!=`

Examples:

```
app = nginx
os != linux
```

Om-Tech

b) Set-based Selectors

- Filter based on a set of values.
- Operators:
  - `in`
  - `notin`
  - `exists`

Examples:

environment in (production, qa)
tier notin (frontend, backend)

Common kubectl Commands

```
kubectl get nodes                          # Check all nodes in the cluster
kubectl apply -f <filename>                # Apply configuration
kubectl get pods                           # List all pods
kubectl get pods -o wide                   # Detailed pod information
kubectl describe <object_type> <name>      # Get detailed info of a resource
kubectl delete <object_type> <name>        # Delete a resource
```

Controllers

Controllers are the core building block of Kubernetes clusters. They're responsible for actioning every object deployed to the cluster, reconciling the desired and current state of cluster resources.

Controller Types

1. ReplicaSet / ReplicationController

A ReplicaSet/ReplicationController ensures that a specified number of replicas of a pod are running at all times.

- If pods exit or are deleted, the replication controller creates new ones to match the desired count.
- If there are more pods than required, it deletes the extra ones.

Om-Tech

Difference:

- ReplicaSet → supports **set-based selectors**
- ReplicationController → supports **equality-based selectors**

---

replicaset.yaml

```
apiVersion: v1
kind: ReplicaSet
metadata:
 name: nginx
spec:
 replicas: 3
 selector:
   app: nginx
 template:
   metadata:
     name: nginx
     labels:
       app: nginx
   spec:
     containers:
       - name: nginx
         image: nginx
         ports:
           - containerPort: 80
```

---

replicationcontroller.yaml

```
apiVersion: v1
kind: ReplicationController
metadata:
 name: nginx
spec:
 replicas: 3
 selector:
   app: nginx
 template:
   metadata:
     name: nginx
     labels:
       app: nginx
   spec:
```

Om-Tech

```
    containers:
      - name: nginx
        image: nginx
        ports:
          - containerPort: 80
```

---

2. Deployment Controller

Deployment is the most common object for deploying the replicas in Kubernetes.

We describe a desired state in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate.

We can define Deployments to:

- Create new ReplicaSets
- Remove existing Deployments and adopt their resources

Deployment provides **declarative updates** for pods without breaking user experience.

We can:

- Roll out updates
- Roll back updates easily

---

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
 labels:
   app: nginx
spec:
 replicas: 3
 selector:
   matchLabels:
     app: nginx
 template:
   metadata:
     labels:
       app: nginx
```

Om-Tech

```
spec:
  containers:
    - name: nginx
      image: nginx:1.14.2
      ports:
        - containerPort: 80
```

Commands

- `kubectl set image <controller> <controller_name> <old_image>=<new_image>`
  → To update the image using deployment controller
- `kubectl rollout history <controller> <controller_name>`
  → To check rollout history
- `kubectl rollout undo <controller> <controller_name>`
  → To rollback to previous version
- `kubectl rollout undo <controller> <controller_name> --to-revision <revision_number>`
  → To rollback to a specific revision

---

3. DaemonSet

DaemonSet is a Kubernetes feature that lets you run a pod on **all cluster nodes** that meet certain criteria.

- When a new node is added → pod is added
- When a node is removed → pod is removed
- Deleting DaemonSet → deletes all its pods

Use cases:

- Logging (e.g., Fluentd)
- Monitoring (e.g., Prometheus agents)
- Node-level services

---

daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
```

Om-Tech

```
metadata:
 name: fluentd-elasticsearch
 namespace: kube-system
 labels:
   k8s-app: fluentd-logging
spec:
 selector:
   matchLabels:
     name: fluentd-elasticsearch
 template:
   metadata:
     labels:
       name: fluentd-elasticsearch
   spec:
     containers:
       - name: fluentd-elasticsearch
         image: image-name
```

---

4. StatefulSet

Stateful Applications

Stateful applications store and track data.

Examples:

- MySQL
- Oracle
- PostgreSQL

Stateless applications do **not store data** and process each request independently.

StatefulSet Overview

A StatefulSet is used to run **stateful applications** in Kubernetes.

Key features:

- Assigns **stable identity** (pod-0, pod-1, pod-2...)
- Pods are created **in order**
- Pods are deleted **in reverse order**
- New pod depends on previous pod being ready

Om-Tech

Example:

- 4 replicas → scale down to 3 → deletes pod-3 first

---

statefulset.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
 name: my-service
spec:
 clusterIP: None
 selector:
   app: my-app
 ports:
   - protocol: TCP
     port: 80
     targetPort: 9376
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: web
spec:
 selector:
   matchLabels:
     app: nginx
 serviceName: "nginx"
 replicas: 3
 minReadySeconds: 10  # by default is 0
 template:
   metadata:
     labels:
       app: nginx
   spec:
     terminationGracePeriodSeconds: 10
     containers:
       - name: nginx
         image: registry.k8s.io/nginx-slim:0.8
         ports:
           - containerPort: 80
             name: web
         volumeMounts:
```

Om-Tech

```
        - name: www
          mountPath: /usr/share/nginx/html
volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: "my-storage-class"
      resources:
        requests:
          storage: 1Gi
```

Pod Lifecycle

The phase of a Pod is a simple, high-level summary of where the Pod is in its lifecycle.

1. **Pending**
   The Pod has been accepted by the Kubernetes system, but one or more of the container images has not been created.
   This includes:
   - Time before scheduling
   - Time spent downloading images
2. **Running**
   The Pod has been bound to a node, and all containers have been created.
   At least one container is running or starting/restarting.
3. **Succeeded**
   All containers in the Pod have terminated successfully and will not restart.
4. **Failed**
   All containers have terminated, and at least one failed (non-zero exit or system termination).
5. **Unknown**
   Pod state could not be determined (usually communication issue with node).

Probes

Probes are used to perform customized health checks in Kubernetes.

- Executed by **kubelet**
- Defined at **container level**
- Help monitor application health

Om-Tech

Probe Configuration

a. **initialDelaySeconds (default=0)**
Delay before first probe runs

b. **periodSeconds (default=10)**
Frequency of probe execution

c. **timeoutSeconds (default=1)**
Max time allowed for probe

d. **successThreshold (default=1)**
Consecutive successes to mark healthy

e. **failureThreshold (default=1)**
Consecutive failures before marking unhealthy

Types of Probes

1. Readiness Probe

- Checks if container is ready to receive traffic
- If it fails → traffic is stopped
- Used when app needs initialization or is overloaded
- Pod is removed from endpoints if failed

---

2. Liveness Probe

- Checks if container is alive
- If failed → kubelet restarts container
- Useful for recovering stuck applications

---

3. Startup Probe

- Used for slow-starting applications
- Disables readiness & liveness until success
- Prevents premature restarts

Om-Tech

Probe Actions / Handlers

i) ExecAction Handler

Runs a command inside the container. Success = exit code 0.

---

exec-liveness.yaml

```
apiVersion: v1
kind: Pod
metadata:
 labels:
   test: liveness
 name: liveness-exec
spec:
 containers:
   - name: liveness
     image: registry.k8s.io/liveness:0.1
     ports:
       - containerPort: 8080
     livenessProbe:
       exec:
         command:
           - cat
           - /usr/share/liveness/html/index.html
       initialDelaySeconds: 5
       periodSeconds: 5
```

---

ii) TCPSocketAction Handler

Attempts TCP connection to pod IP and port. Success if port is open.

---

tcp-liveness.yaml

```
apiVersion: v1
kind: Pod
metadata:
 name: liveness
```

Om-Tech

```
 labels:
   app: liveness-tcp
spec:
 containers:
   - name: liveness
     image: registry.k8s.io/liveness:0.1
     ports:
       - containerPort: 8080
     livenessProbe:
       tcpSocket:
         port: 8080
       initialDelaySeconds: 5
       periodSeconds: 5
```

---

iii) HTTPGetAction Handler

Performs HTTP GET request. Success if response code **200–399**.

---

http-liveness.yaml

```
apiVersion: v1
kind: Pod
metadata:
 labels:
   test: liveness
 name: liveness-http
spec:
 containers:
   - name: liveness
     image: registry.k8s.io/liveness:0.1
     livenessProbe:
       httpGet:
         path: /health
         port: 8080
         httpHeaders:
           - name: Custom-Header
             value: ItsAlive
       initialDelaySeconds: 5
       periodSeconds: 5
```

---

iv) gRPC Handler

Om-Tech

- Available from Kubernetes **v1.24+**
- Uses gRPC health checking protocol
- Must enable **GRPCContainerProbe feature gate**

---

grpc-liveness.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
 labels:
   test: liveness
 name: liveness-gRPC
spec:
 containers:
   - name: liveness
     image: registry.k8s.io/liveness:0.1
     ports:
       - containerPort: 2379
     livenessProbe:
       grpc:
         port: 2379
       initialDelaySeconds: 5
       periodSeconds: 5
```

Kubernetes Services

Services in Kubernetes are a way of defining network configuration for pods.

Services always point to pods, deployments, or other controllers.

In Kubernetes, pods can communicate with each other by default using internal IP addresses assigned by kube-proxy.

Need for Kubernetes Services

a. Pods are short-lived. When a pod dies and a new pod comes up, its IP changes, so we cannot rely on internal IPs.

Om-Tech

b. Pods can scale horizontally, and each new pod gets its own IP, which cannot be predicted.

c. Pod IPs cannot be used directly when exposing services outside the cluster.

Types of Services in Kubernetes

1. ClusterIP

- Default service type
- Enables communication within the cluster
- Not accessible from outside

Use case: Internal communication (e.g., frontend → backend)

---

clusterip.yaml

```
apiVersion: v1
kind: Service
metadata:
 name: backend
spec:
 type: ClusterIP
 ports:
   - name: http
     protocol: TCP
     port: 80
     targetPort: 8080
```

---

2. NodePort

- Extension of ClusterIP
- Exposes service on each node's IP
- Port range: **30000–32767**

Use case: External access without cloud LB

---

nodeport.yaml

```
apiVersion: v1
```

Om-Tech

```yaml
kind: Service
metadata:
 name: frontend
spec:
 type: NodePort
 selector:
   app: web
 ports:
   - name: http
     protocol: TCP
     port: 80
     targetPort: 8080
     nodePort: 30000
```

---

3. LoadBalancer

- Extension of NodePort
- Creates a cloud load balancer
- Automatically routes traffic to service

Use case: Cloud environments (AWS, Azure, GCP)

---

loadbalancer.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
 name: frontend
spec:
 type: LoadBalancer
 selector:
   app: web
 ports:
   - name: http
     protocol: TCP
     port: 80
     targetPort: 8080
```

---

4. Headless Service

- No cluster IP (`clusterIP: None`)

Om-Tech

- Direct pod access (no load balancing)

Use case: Stateful apps, custom load balancing

---

headless.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
 name: DB
spec:
 clusterIP: None
 selector:
   app: DB-app
 ports:
   - protocol: TCP
     port: 80
     targetPort: 9376
```

Scheduling Pods

By default, scheduling is handled by the Kubernetes scheduler.

The scheduler:

- Checks node availability
- Selects the best node for each pod

Why Manual Scheduling?

a. To run pods on nodes with specific resources

b. To place pods in a specific availability zone

c. To co-locate dependent services on same node

Commands

Om-Tech

- `kubectl get nodes --show-labels` → Show node labels
- `kubectl label node <node_name> <key>=<value>` → Add label
- `kubectl label node <node_name> <key>-` → Remove label

Manual Scheduling Methods

1. Node Selector

Simplest method to schedule pods using node labels.

---

nodeselector.yaml

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
   env: test
spec:
 containers:
   - name: nginx
     image: nginx
     imagePullPolicy: IfNotPresent
 nodeSelector:
   disktype: ssd
```

---

2. Affinity and Anti-Affinity

More advanced scheduling than nodeSelector.

Benefits

a. More expressive rules

b. Supports soft (preferred) and hard (required) rules

c. Can use pod labels (not just node labels)

Om-Tech

Types of Affinity

i) Node Affinity

Similar to nodeSelector but more flexible.

- **requiredDuringSchedulingIgnoredDuringExecution** → Hard rule
- **preferredDuringSchedulingIgnoredDuringExecution** → Soft rule

---

nodeaffinity.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
 name: with-node-affinity
spec:
 affinity:
   nodeAffinity:
     requiredDuringSchedulingIgnoredDuringExecution:
       nodeSelectorTerms:
         - matchExpressions:
             - key: topology.kubernetes.io/zone
               operator: In
               values:
                 - ap-south-1
     preferredDuringSchedulingIgnoredDuringExecution:
       - weight: 1
         preference:
           matchExpressions:
             - key: disktype
               operator: In
               values:
                 - ssd
 containers:
   - name: with-node-affinity
     image: registry.k8s.io/pause:2.0
```

Node Affinity Weight

- Range: **1–100**

Om-Tech

- Higher weight → higher priority node

Scheduler sums weights and selects best node.

---

pod-with-affinity-anti-affinity.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
 name: with-affinity-anti-affinity
spec:
 affinity:
   nodeAffinity:
     requiredDuringSchedulingIgnoredDuringExecution:
       nodeSelectorTerms:
         - matchExpressions:
             - key: kubernetes.io/os
               operator: In
               values:
                 - linux
     preferredDuringSchedulingIgnoredDuringExecution:
       - weight: 1
         preference:
           matchExpressions:
             - key: label-1
               operator: In
               values:
                 - key-1
       - weight: 50
         preference:
           matchExpressions:
             - key: label-2
               operator: In
               values:
                 - key-2
 containers:
   - name: with-node-affinity
     image: registry.k8s.io/pause:2.0
```

---

ii) Inter-Pod Affinity and Anti-Affinity

- Pod affinity → co-locate pods

Om-Tech

- Pod anti-affinity → avoid placing pods together

---

podaffinity.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
 name: with-pod-affinity
spec:
 affinity:
   podAffinity:
     requiredDuringSchedulingIgnoredDuringExecution:
       - labelSelector:
           matchExpressions:
             - key: security
               operator: In
               values:
                 - S1
         topologyKey: topology.kubernetes.io/zone
   podAntiAffinity:
     preferredDuringSchedulingIgnoredDuringExecution:
       - weight: 100
         podAffinityTerm:
           labelSelector:
             matchExpressions:
               - key: security
                 operator: In
                 values:
                   - S2
           topologyKey: topology.kubernetes.io/zone
 containers:
   - name: with-pod-affinity
     image: registry.k8s.io/pause:2.0
```

---

3. Taints and Tolerations

- **Affinity** → attracts pods to nodes
- **Taints** → repel pods from nodes
- **Tolerations** → allow pods to tolerate taints

Om-Tech

Commands

- `kubectl taint nodes node1 key1=value1:NoSchedule` → Add taint
- `kubectl taint nodes node1 key1=value1:NoSchedule-` → Remove taint

Tolerations Example

```
tolerations:
 - key: "key1"
   operator: "Equal"
   value: "value1"
   effect: "NoSchedule"
```

OR

```
tolerations:
 - key: "key1"
   operator: "Exists"
   effect: "NoSchedule"
```

---

toleration.yaml

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
   env: test
spec:
 containers:
   - name: nginx
     image: nginx
     imagePullPolicy: IfNotPresent
 tolerations:
   - key: "example-key"
     operator: "Exists"
     effect: "NoSchedule"
```

Taint Effects

a. **NoSchedule**

Om-Tech

- New pods won't be scheduled unless tolerated
- Existing pods remain

b. **NoExecute**

- Existing pods are evicted
- New pods are not scheduled

c. **PreferNoSchedule**

- Soft rule → scheduler tries to avoid placing pods

Kubernetes Volumes

A Kubernetes volume is a directory containing data accessible to containers in a given pod, the smallest deployable unit in a Kubernetes cluster. Within the Kubernetes container orchestration and management platform, volumes provide a plugin mechanism that connects ephemeral containers with persistent data storage.

Types of Volumes

o Ephemeral Volumes:
Ephemeral volume types have a lifetime of a pod. When a pod ceases to exist, Kubernetes destroys ephemeral volumes.

o Persistent Volumes:
Kubernetes Persistent Volumes (PVs) provide storage for your application's Pods. Data written to a volume is managed independently of the Pods that access it, ensuring the data remains available after Pod restarts and failures.

You'll need to use PVs whenever you deploy stateful applications such as databases and file servers using Kubernetes. Without a PV, your data will be lost when Pods are terminated.

Persistent Volumes are Kubernetes objects that represent storage resources in your cluster. PVs work in conjunction with Persistent Volume Claims (PVCs), another type of object which permits Pods to request access to PVs. To successfully utilize persistent storage in a cluster, you'll need a PV and a PVC that connects it to your Pod.

- **o static** – Manually creating the persistent volumes and claimed using PVC.
  example: hostpath
- **o Dynamic** – We configure storage classes to dynamically create persistent volumes. Volumes are created dynamically on demand.

Om-Tech

example: CSI – Allows integration with storage providers that support the Container Storage Interface (CSI) specification, such as the block storage services provided by cloud platforms.

PersistentVolumeClaim

A persistentVolumeClaim volume is used to mount a PersistentVolume into a Pod. PersistentVolumeClaims are a way for users to "claim" durable storage.

Persistent Volume Access Modes

a. **ReadWriteOnce (RWO)** – The volume is mounted with read-write access for a single Node in your cluster. Any of the Pods running on that Node can read and write the volume's contents.

b. **ReadOnlyMany (ROX)** – The volume can be concurrently mounted to any of the Nodes in your cluster, with read-only access for any Pod.

c. **ReadWriteMany (RWX)** – Similar to ReadOnlyMany, but with read-write access.

d. **ReadWriteOncePod (RWOP)** – This new variant, introduced as a beta feature in Kubernetes v1.27, enforces that read-write access is provided to a single Pod. No other Pods in the cluster will be able to use the volume simultaneously.

Reclaim Policies

a. **Retain:**
When the PersistentVolumeClaim is deleted, the PersistentVolume will still exist, and the volume is considered "released". But it is not yet available for another claim because the previous claimant's data remains on the volume. An administrator must manually reclaim the volume.

b. **Delete:**
Delete reclaim policy removes both the PersistentVolume object from Kubernetes, as well as the associated storage.

c. **Recycle:**
Recycle reclaim policy performs a basic scrub (`rm -rf /thevolume/*`) on the volume and makes it available again for a new claim.

Om-Tech

Kubernetes Namespace

Kubernetes Namespace is a mechanism that enables you to organize resources. It is like a virtual cluster inside the cluster. A namespace isolates the resources from the resources of other namespaces.

For example, you need to have different names for deployments/services in a namespace, but you can have the same name for deployment in two different namespaces.

Initial Namespaces

a. **default** – Kubernetes includes this namespace so that you can start using your new cluster without first creating a namespace.

b. **kube-node-lease** – This namespace holds Lease objects associated with each node. Node leases allow the kubelet to send heartbeats so that the control plane can detect node failure.

c. **kube-public** – This namespace is readable by all clients (including those not authenticated). This namespace is mostly reserved for cluster usage, in case that some resources should be visible and readable publicly throughout the whole cluster. The public aspect of this namespace is only a convention, not a requirement.

d. **kube-system** – This is the namespace that you use to create your resources by default.

ConfigMaps

ConfigMap is a Kubernetes resource that decouples configuration from pods and other Kubernetes resources.

A ConfigMap is an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as:

- environment variables
- command-line arguments
- configuration files in a volume

One thing to keep in mind while using ConfigMap is that you must create it first and then create the resource that references the ConfigMap; otherwise, the workload wouldn't start.

Om-Tech

Also, make sure that the key you're passing to the workload is not misspelled and exists in the ConfigMap.

They also have a size limitation of **1MB**.

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: db-url
data:
 DBURL: postgres.cwdlsh.ap-south-1.rds.com
```

CLI

```
kubectl create cm db-url --from-literal DBURL=postgres.cwdlsh.ap-south-1.rds.com
```

Secrets

Kubernetes Secrets are designed to store and manage sensitive information such as:

- tokens
- passwords
- SSH keys

Instead of storing this information within an application or its associated source code, it is stored in Secrets to enhance security.

Types of Secrets

- Opaque
- kubernetes.io/service-account-token
- kubernetes.io/dockercfg
- kubernetes.io/dockerconfigjson
- kubernetes.io/basic-auth
- kubernetes.io/ssh-auth
- kubernetes.io/tls

Every Secret holds:

- **data** (base64 encoded)
- **stringData** (write-only, auto-encoded)

Om-Tech

Secrets are:

- stored in etcd
- transmitted securely via API server
- optionally encrypted at rest

---

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
 name: db-password
data:
 DBPASSWORD: ZGV2b3BzQDEyMzQK
```

CLI

```
kubectl create secret genric db-password --from-literal DBPASSWORD=devops@123
```

Secret vs ConfigMap

- Both are stored in etcd
- Both have a size limit of 1MB
- Both have similar lifecycle

Difference:

- Secrets → Sensitive data (encrypted)
- ConfigMaps → Non-sensitive data (not encrypted)

ResourceQuota

Resource Quotas in Kubernetes allow administrators to set constraints on the total resource usage per namespace.

This helps in preventing a single namespace from monopolizing cluster resources.

Om-Tech

Use Cases

1. Ability to enumerate resource usage limits per namespace
2. Ability to monitor resource usage for tracked resources
3. Ability to reject resource usage exceeding hard quotas

---

resourcequota.yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
 name: rq
 namespace: dev
spec:
 hard:
   pods: "1"
   requests.cpu: "2"
   requests.memory: "5Gi"
   limits.cpu: "4"
   limits.memory: "10Gi"
```

Securing Kubernetes

Securing Kubernetes requires a multi-layered approach that addresses the **Four Cs: Cloud, Cluster, Container, and Code**.

Start by keeping your Kubernetes environment up to date with the latest stable versions to take advantage of security patches and improvements. Secure API server access with encrypted communication and robust authentication methods like certificates or third-party identity providers. Enable role-based access control (RBAC) to enforce least-privilege access for users and services, and avoid unnecessary permissions.

Harden nodes by minimizing administrative access and isolating sensitive workloads using node selectors and namespaces. Control network traffic between pods with network policies and monitor cluster activity through detailed audit logging to detect suspicious behavior. Finally, continuously scan containers for vulnerabilities, integrate security into the CI/CD pipeline, and automate security controls to stay ahead of evolving threats.

Om-Tech

Key Security Practices

- Upgrade Kubernetes to latest version
- Secure Kubernetes API server authentication
- Enable role-based access control authorization
- Control access to the kubelet
- Harden node security
- Set up namespaces and network policies
- Enable audit logging

RBAC (Role-Based Access Control)

RBAC or Role-Based Access Control is an approach in Kubernetes used to add constraints for users, groups, and applications to access Kubernetes resources.

RBAC basically adds security to the Kubernetes cluster and can be applied to a specific namespace or the entire cluster. It was introduced in version 1.8 and uses the `rbac.authorization.k8s.io` API group.

RBAC is all about managing access to resources. It's not about identifying a specific user or authenticating them. With lots of people and resources, it's difficult to manage an inventory of who can access what. So an automated system is needed to allow or deny access to resources.

Benefits of RBAC

- Reduced admin load as RBAC reduces manual redundant access provisioning
- Enhanced compliance through automation with a defined ruleset
- Reduced risk of breach by minimizing human error

Three Important Concepts in RBAC

- **Subject**: Entity that needs access (user, group, or service account)
- **Resources**: Kubernetes objects (pods, deployments, services, etc.)
- **Verbs**: Actions on resources (get, list, create, delete, watch, etc.)

RBAC Objects

- **Role**: Grants access to resources within a specific namespace
- **ClusterRole**: Grants access across the entire cluster
- **RoleBinding**: Binds a Role to a Subject within a namespace
- **ClusterRoleBinding**: Binds a ClusterRole to a Subject cluster-wide

RBAC Flow in Kubernetes

a. The User authenticates with Kubernetes
b. Kubernetes generates an authorization token and sends it to the User
c. The User tries to access resources
d. Kubernetes checks the permissions using RBAC
e. RBAC decides whether access is allowed
f. Kubernetes informs the User whether access is granted or denied

---

Service Account

A service account is a type of non-human account that provides a distinct identity in a Kubernetes cluster.

Application Pods, system components, and entities inside and outside the cluster can use a ServiceAccount's credentials to identify themselves. This is useful for authenticating to the API server or implementing identity-based security policies.

Service accounts exist as objects in the API server.

Properties of Service Accounts

- **Namespaced**: Each service account belongs to a namespace; every namespace gets a default service account
- **Lightweight**: Easily created and managed through the Kubernetes API
- **Portable**: Can be included in configuration bundles for workloads

Service accounts differ from user accounts. User accounts are authenticated human users and are not stored in the Kubernetes API server by default. Instead, identities are treated as opaque data and authenticated through external methods.

Context

A context is a named set of access parameters that defines how to communicate with a Kubernetes cluster.

It includes:

Om-Tech

- Cluster endpoint
- Authentication details
- Default namespace

Contexts act like snapshots of Kubernetes environments and are essential when working with multiple clusters (e.g., dev, staging, production).

Commands

kubectl config get-contexts

→ Displays available contexts

kubectl config use-context <context-name>

→ Switches between contexts

.kubeconfig File

A `.kubeconfig` file is a YAML configuration file used by `kubectl` to manage access to Kubernetes clusters.

Main Sections

- **clusters**: Cluster details and server endpoints
- **users**: Authentication details (certificates, tokens)
- **contexts**: Mapping of cluster, user, and namespace
- **current-context**: Default context

YAML Configurations

Role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: node-viewer
rules:
```

Om-Tech

```
- apiGroups: [""]
 resources: ["nodes"]
 verbs: ["get", "list"]
```

---

rolebinding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: node-viewer-binding
subjects:
- kind: Group
 name: ops-team
 apiGroup: rbac.authorization.k8s.io
roleRef:
 kind: ClusterRole
 name: node-viewer
 apiGroup: rbac.authorization.k8s.io
```

---

clusterrole.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: sa-cr
rules:
- apiGroups: ["*"]
 resources: ["pods", "services", "secrets"]
 verbs: ["get", "create", "list"]
```

---

clusterrolebinding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: sa-crb
subjects:
- kind: ServiceAccount
 name: my-sa
 namespace: default
roleRef:
```

Om-Tech

```
kind: ClusterRole
name: sa-cr
apiGroup: rbac.authorization.k8s.io
```

Om-Tech